

# RAG for Invoice Processing - Concepts and Case Study

**Naresh Chaurasia**

Version 1.0, 12/Jun/2026: First Edition

# Table of Contents

|                                                                  |    |
|------------------------------------------------------------------|----|
| 1. The Problem: Manual Invoice Processing .....                  | 1  |
| 1.1. What Happens Today .....                                    | 1  |
| 1.2. Why This Breaks Down .....                                  | 1  |
| 1.3. The Scale of the Problem .....                              | 2  |
| 2. What RAG Is .....                                             | 3  |
| 2.1. A Simple Analogy .....                                      | 3  |
| 2.2. Why Not Just Use a Simple Search? .....                     | 3  |
| 3. How RAG Solves the Invoice Problem .....                      | 4  |
| 3.1. Step 1: Reading and Storing the Invoice .....               | 4  |
| 3.2. Step 2: Asking a Question About the Invoice .....           | 4  |
| 3.3. Step 3: The Language Model Produces the Answer .....        | 4  |
| 3.4. The Contrast with Manual Processing .....                   | 4  |
| 4. The RAG Architecture .....                                    | 6  |
| 4.1. Overview: The Two Pipelines .....                           | 6  |
| 4.2. Pipeline 1: The Ingestion Pipeline .....                    | 6  |
| 4.2.1. Stage 1: Text Extraction .....                            | 7  |
| 4.2.2. Stage 2: Chunking .....                                   | 7  |
| 4.2.3. Stage 3: Embedding .....                                  | 7  |
| 4.2.4. Stage 4: Storage in the Vector Database .....             | 8  |
| 4.3. Pipeline 2: The Query Pipeline .....                        | 8  |
| 4.3.1. Stage 1: Query Embedding .....                            | 9  |
| 4.3.2. Stage 2: Similarity Search in the Vector Database .....   | 9  |
| 4.3.3. Stage 3: Prompt Construction .....                        | 9  |
| 4.3.4. Stage 4: LLM Generation .....                             | 10 |
| 4.3.5. Stage 5: Structured Output .....                          | 10 |
| 4.4. The Complete End-to-End Picture .....                       | 10 |
| 5. The Technologies Behind RAG .....                             | 11 |
| 5.1. Large Language Models .....                                 | 11 |
| 5.2. Vector Databases .....                                      | 11 |
| 5.3. LangChain .....                                             | 11 |
| 6. Course Learning Path .....                                    | 12 |
| 6.1. Lesson 1: Understanding Language Models .....               | 12 |
| 6.2. Lesson 2: RAG and Vector Databases .....                    | 12 |
| 6.3. Lesson 3: Hands-On Projects and Real-World Deployment ..... | 12 |
| 7. Practical Outcomes .....                                      | 13 |
| 7.1. Automating Structured Data Extraction .....                 | 13 |
| 7.2. Building Intelligent Knowledge Retrieval Chatbots .....     | 13 |
| 7.3. Deploying Real-World Business Process Automation .....      | 13 |

|                                     |    |
|-------------------------------------|----|
| 8. Summary .....                    | 14 |
| 9. Quick Self-Check Questions ..... | 15 |
| 9.1. Foundational Level .....       | 15 |
| 9.2. Intermediate Level .....       | 15 |

# Chapter 1. The Problem: Manual Invoice Processing

## 1.1. What Happens Today

Every business receives invoices from its suppliers. An invoice is a document that says: here is what we delivered, here is the price, please pay by this date. A small company might receive 50 invoices a month. A mid-sized manufacturing company might receive 5,000.

Each invoice arrives as a PDF, a scanned image, or sometimes a Word document. A human employee opens each file, reads it, types the key information into the company's accounting system, and files the document.

The key information that must be extracted includes:

- Invoice number and date
- Vendor name and tax identification number
- List of items purchased, with quantities and prices
- Subtotal, taxes, and total amount due
- Payment due date

## 1.2. Why This Breaks Down

The core difficulty is that every supplier uses a different layout. Consider two suppliers providing the same product.

**Supplier A — TechParts Ltd. (Mumbai)** formats their invoice like this:

```
INVOICE
Invoice No: TP-2024-00892      Date: 05 Jun 2026
Vendor: TechParts Ltd.
Bill To: Acme Manufacturing

Description      Qty    Unit Price    Amount
-----
Servo Motor 24V    10    INR 4,500    INR 45,000
Control Board Rev3    5    INR 2,200    INR 11,000
-----
Subtotal                      INR 56,000
GST 18%                      INR 10,080
Total Due                    INR 66,080
Payment Due: 05 Jul 2026
```

**Supplier B — Elektro GmbH (Germany)** formats their invoice like this:

Rechnung Nr.: EG-8812                      Datum: 05.06.2026  
Lieferant: Elektro GmbH  
Rechnungsempfänger: Acme Manufacturing

| Pos.                     | Bezeichnung        | Menge | Einzelpreis | Gesamt     |
|--------------------------|--------------------|-------|-------------|------------|
| 1                        | Servo-Motor 24V    | 10    | EUR 48.00   | EUR 480.00 |
| 2                        | Steuerplatine Rev3 | 5     | EUR 23.50   | EUR 117.50 |
| -----                    |                    |       |             |            |
| Nettobetrag              |                    |       |             | EUR 597.50 |
| MwSt 19%                 |                    |       |             | EUR 113.53 |
| Rechnungsbetrag          |                    |       |             | EUR 711.03 |
| Zahlungsziel: 05.07.2026 |                    |       |             |            |

Both invoices carry the same type of information, but the field names are different, the language is different, the date format is different, and the column layout is different. A rule that looks for the word "Total Due" would find the amount in the first invoice but would fail completely on the second, which uses "Rechnungsbetrag".

## 1.3. The Scale of the Problem

At 5,000 invoices a month, with an average of 8 minutes of human effort per invoice, the company spends roughly 667 person-hours on data entry alone every month. That is the equivalent of four full-time employees doing nothing but reading and typing.

Beyond the cost, there is the error rate. Human keying errors — transposed digits, missed fields, wrong dates — occur on approximately 1 in 100 invoices. At 5,000 invoices, that is 50 errors per month entering the accounting system, each requiring investigation and correction.

When business volume grows, the only traditional solution is to hire more people. The cost scales directly with volume, which destroys the economics of growth.

# Chapter 2. What RAG Is

## 2.1. A Simple Analogy

Imagine you hire a new employee who is extremely intelligent and can read and understand any language. On their first day you hand them an invoice they have never seen before and say: "Read this and fill in this form." They read the document, understand it regardless of its layout or language, and correctly fill in every field on the form.

Now imagine that same employee can process one invoice every three seconds, around the clock, without ever making a typing error. That is what a RAG-powered system does.

RAG stands for Retrieval-Augmented Generation. The name describes two things that happen together:

- **Retrieval** means finding the right piece of information from a document or a collection of documents. The system does not read everything blindly; it locates the most relevant sections first.
- **Generation** means using a Large Language Model (an AI that understands language) to read the retrieved content and produce a structured, accurate answer.

The combination allows the system to handle any document format, any language, and any field naming convention without needing a custom rule for each supplier.

## 2.2. Why Not Just Use a Simple Search?

A traditional keyword search looks for exact words. If you search for "Total Due" and the invoice says "Rechnungsbetrag", the search returns nothing. The two phrases mean the same thing but share no common words.

RAG uses a different approach called semantic search. Instead of matching words, it matches meaning. The system converts both "Total Due" and "Rechnungsbetrag" into numerical representations of their meaning, called vectors or embeddings. Because both phrases mean "the amount the buyer owes", their numerical representations are close together, and the system recognizes them as related even though the words are completely different.

# Chapter 3. How RAG Solves the Invoice Problem

## 3.1. Step 1: Reading and Storing the Invoice

When an invoice arrives, the system reads its entire text and breaks it into small overlapping sections called chunks. Each chunk is converted into a numerical representation of its meaning and stored in a special database called a vector database.

Think of it this way: the system reads the invoice once, understands what each section is about, and stores a summary of the meaning of every part. The original text is also kept, but the numerical meaning allows fast searching later.

## 3.2. Step 2: Asking a Question About the Invoice

The system then asks itself a question, such as: "What is the total amount owed on this invoice, including all taxes?"

This question is also converted into a numerical representation of its meaning. The system then searches the vector database for the chunks whose meaning is closest to the meaning of the question. For the TechParts invoice, it would retrieve the section containing "Total Due INR 66,080". For the Elektro GmbH invoice, it would retrieve the section containing "Rechnungsbetrag EUR 711.03", because both sections mean the same thing in terms of their purpose in the document.

## 3.3. Step 3: The Language Model Produces the Answer

The retrieved sections are handed to a Large Language Model together with an instruction such as: "Read the text below, which is from an invoice, and extract the total amount due and its currency."

The language model reads the text, understands it in whatever language it is written, and returns a clean, structured answer. It handles the German text just as well as the English text because modern language models are trained on text in dozens of languages.

The output is no longer a raw document. It is structured data:

- Invoice number: TP-2024-00892
- Total due: 66,080
- Currency: INR
- Payment due: 2026-07-05

This structured data goes directly into the accounting system without any human typing.

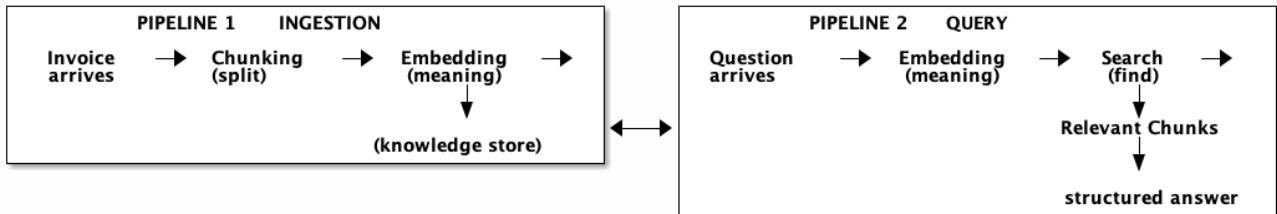
## 3.4. The Contrast with Manual Processing

| Aspect            | Manual Processing                                                    | RAG-Powered Processing                                                   |
|-------------------|----------------------------------------------------------------------|--------------------------------------------------------------------------|
| Time per invoice  | 6 to 10 minutes of human reading and typing                          | 2 to 4 seconds of automated processing                                   |
| Format handling   | Employee adapts mentally; requires training for new supplier formats | System handles any format automatically; no retraining needed            |
| Language handling | Requires a bilingual employee for foreign invoices                   | System reads any language the model was trained on                       |
| Error rate        | Approximately 1% due to fatigue, distraction, and typos              | Near zero for well-structured invoices; errors are flagged automatically |
| Scalability       | Adding volume requires hiring more staff                             | Volume can increase tenfold with no additional cost                      |
| Availability      | Limited to working hours                                             | Operates continuously, 24 hours a day                                    |

# Chapter 4. The RAG Architecture

RAG has two distinct pipelines that work in sequence. The first pipeline runs once per document and prepares the knowledge base. The second pipeline runs every time a user asks a question. Understanding both pipelines is essential to understanding how the system works end to end.

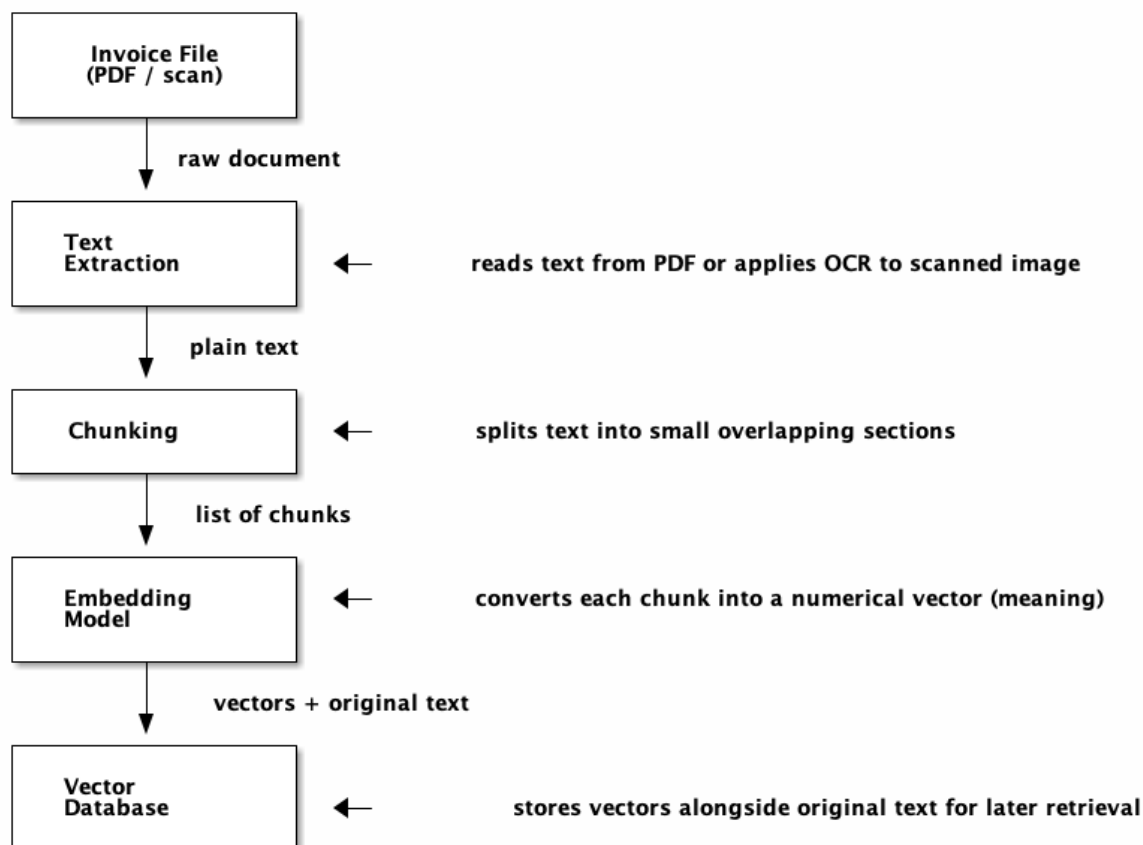
## 4.1. Overview: The Two Pipelines



The ingestion pipeline runs when a new invoice is uploaded. The query pipeline runs when the accounts-payable team asks for data to be extracted or a question to be answered. Both pipelines share the vector database as their meeting point.

## 4.2. Pipeline 1: The Ingestion Pipeline

The ingestion pipeline transforms a raw invoice document into searchable, meaningful data stored in the vector database. It has four stages.



#### 4.2.1. Stage 1: Text Extraction

The invoice arrives as a file. If it is a digitally generated PDF, the text layer can be read directly. If it is a scanned image, Optical Character Recognition (OCR) is applied first to convert the image into readable text. The output of this stage is the full plain text of the invoice.

For the TechParts invoice, the extracted text would include every line: the invoice number, the date, the vendor name, the line items, the totals, and the payment terms.

#### 4.2.2. Stage 2: Chunking

A language model has a limit on how much text it can read in one request, called the context window. More importantly, when searching for a specific piece of information, it is more precise to search within small focused sections than to search the entire document at once.

Chunking splits the invoice text into small overlapping sections. A typical chunk might be 300 to 500 words. The overlap (repeating a small portion of text between consecutive chunks) ensures that information sitting at the boundary between two chunks is not lost.

For a short single-page invoice, chunking might produce two or three chunks. For a ten-page contract with many line items, it might produce twenty or thirty chunks.

#### 4.2.3. Stage 3: Embedding

Each chunk is passed to an embedding model. The embedding model is a specialized AI whose sole

job is to convert text into a vector: an ordered list of hundreds or thousands of numbers. This vector encodes the meaning of the text.

Two chunks that discuss the same concept—for example, "Total Due INR 66,080" and "Rechnungsbetrag EUR 711.03"—will produce vectors that are numerically close to each other, because both express the concept of the final amount payable. Two chunks that discuss unrelated concepts—for example, a payment total and a vendor address—will produce vectors that are far apart.

This numerical closeness is what makes semantic search possible.

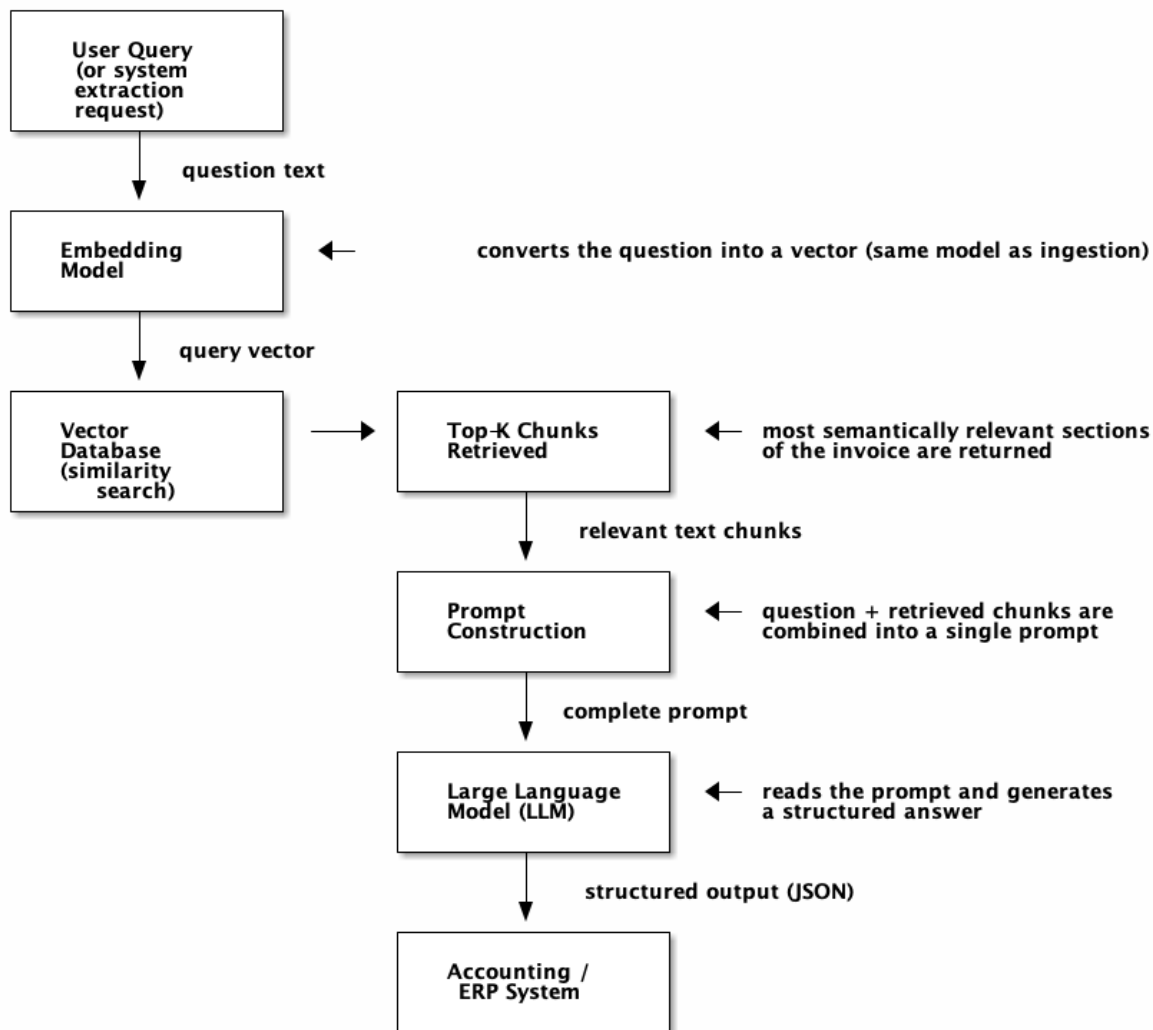
#### **4.2.4. Stage 4: Storage in the Vector Database**

The vector for each chunk is stored in the vector database alongside the original text of that chunk. The database is designed to answer one question very quickly: given a query vector, which stored vectors are closest to it?

After ingestion, the invoice is fully indexed. Nothing more needs to happen until someone asks a question.

### **4.3. Pipeline 2: The Query Pipeline**

The query pipeline runs every time the system needs to extract information from an invoice or answer a question about it. It has five stages.



#### 4.3.1. Stage 1: Query Embedding

The question—for example, "What is the total amount due on this invoice?"—is passed through the same embedding model used during ingestion. This converts the question into a vector using the same numerical space. The model must be the same in both pipelines; otherwise the vectors would be incomparable.

#### 4.3.2. Stage 2: Similarity Search in the Vector Database

The query vector is compared against all stored chunk vectors. The database returns the top-K chunks: the K chunks whose vectors are closest to the query vector. K is typically 3 to 5. These chunks contain the invoice sections most relevant to the question.

For the question about the total amount, the database would return the chunk containing the totals section of the invoice, because the meaning of "total amount due" is closest to the meaning of "Total Due INR 66,080" or "Rechnungsbetrag EUR 711.03".

#### 4.3.3. Stage 3: Prompt Construction

The retrieved chunks and the original question are assembled into a single prompt. The prompt gives the language model all the context it needs: a clear instruction, the relevant sections of the

invoice, and the required output format.

A typical prompt reads: "You are an accounts-payable assistant. Read the invoice sections below and extract the specified fields. Return your answer as a JSON object." The invoice chunks are appended beneath this instruction.

#### 4.3.4. Stage 4: LLM Generation

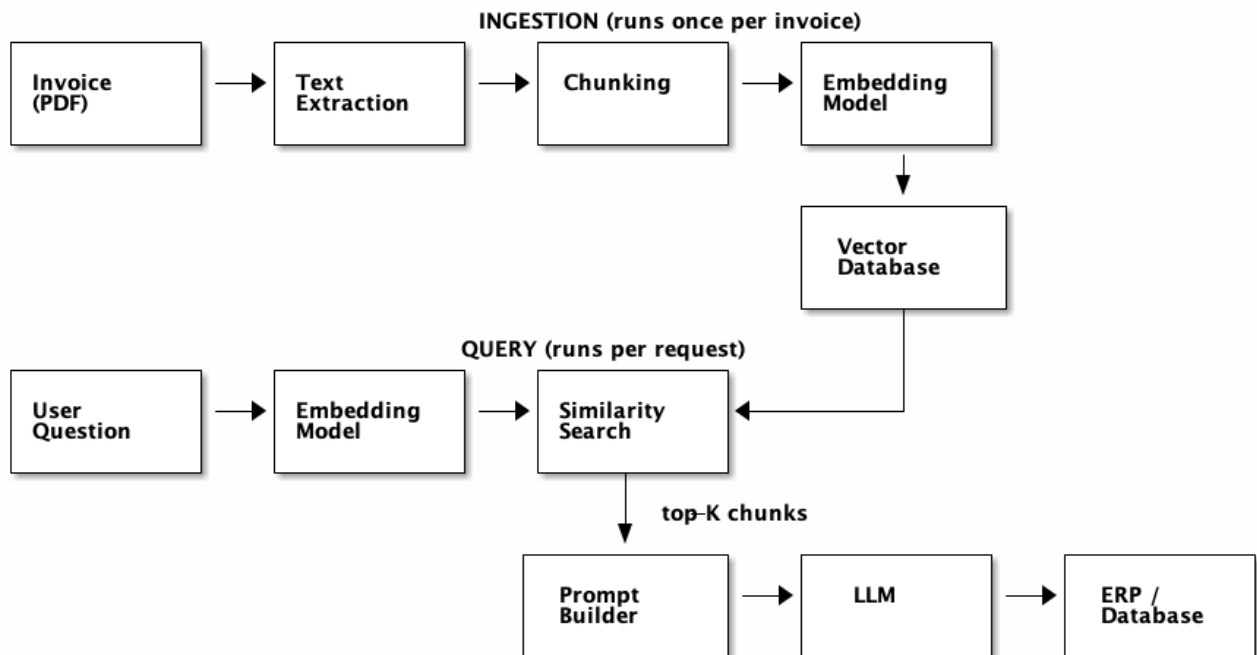
The language model reads the complete prompt and generates a response. It understands the content in any language, interprets the structure of the invoice, and produces the requested output in the specified format.

The key point is that the model is not guessing or searching its training data for facts about this invoice. It is reading the actual invoice text that was retrieved and provided to it. This is what makes RAG reliable: the model always works from the source document.

#### 4.3.5. Stage 5: Structured Output

The model's response is a structured JSON object. This object is validated — for example, checking that line item totals add up to the subtotal, and that dates are in a recognizable format — and then written directly into the accounting or ERP system.

### 4.4. The Complete End-to-End Picture



The ingestion pipeline and the query pipeline share only the vector database. Every other component operates independently. This separation means invoices can be ingested in the background continuously, and queries can be answered in real time without waiting for ingestion to complete.

# Chapter 5. The Technologies Behind RAG

## 5.1. Large Language Models

A Large Language Model is an AI system trained on enormous amounts of text — books, articles, websites, technical documents — in many languages. Through this training, the model learns to understand the meaning and structure of language with remarkable depth.

When given an invoice, the model does not merely pattern-match on keywords. It understands that "Rechnungsbetrag", "Total Due", "Montant Total", and "Importe Total" all refer to the same concept. It understands that a number following a currency symbol is a monetary amount. It understands the relationship between line items and totals.

Examples of widely used language models include OpenAI GPT-4o and Google Gemini. These models are accessed over the internet via an API, meaning the application sends the invoice text to the model and receives the structured extraction back.

## 5.2. Vector Databases

A vector database is a specialized storage system designed to hold and search numerical representations of meaning. When the invoice is ingested, every section of the document is converted to a vector — an array of hundreds or thousands of numbers that encodes what that section means.

When a query arrives, it is also converted to a vector. The database finds the stored vectors that are numerically closest to the query vector. Closeness in the numerical space corresponds to similarity in meaning.

Popular vector databases include Pinecone, Weaviate, and ChromaDB. Each stores the vectors alongside the original text so that when a relevant section is found, the original words can be retrieved and sent to the language model.

## 5.3. LangChain

LangChain is a framework that connects these components into a working pipeline. It provides ready-made connectors for loading PDFs, splitting them into chunks, calling embedding models, storing in vector databases, constructing prompts, calling language models, and parsing their responses.

Rather than building each connection from scratch, a developer uses LangChain to assemble the pipeline from pre-built, tested components. This is the same role that a web framework plays in web development — it handles the plumbing so the developer can focus on the business logic.

# Chapter 6. Course Learning Path

## 6.1. Lesson 1: Understanding Language Models

The first lesson builds a foundation for everything that follows. Students learn what a language model is and how it processes text. They learn the concept of tokens, which are the units the model reads and generates. They understand the context window, which sets the limit on how much text the model can consider at once.

Most importantly, students learn prompt design: how to write clear, precise instructions that guide the model to produce the output you need. A poorly written prompt produces vague, inconsistent results. A well-written prompt produces reliable, parseable output every time. Using the invoice example, students practise writing prompts that instruct the model to extract specific fields and return them in a specific format.

## 6.2. Lesson 2: RAG and Vector Databases

The second lesson introduces the retrieval layer. Students learn why a language model alone is not sufficient when dealing with large collections of documents—the context window has limits, and sending an entire archive of invoices to the model in one request is not practical.

Students learn what embeddings are, how meaning is encoded as numbers, and how vector databases make similarity search fast and scalable. They learn the two phases of the RAG pipeline: the ingestion phase, where documents are read, chunked, and stored; and the query phase, where a question retrieves relevant chunks and passes them to the model.

Using the invoice scenario, students trace the path of a single invoice from upload to structured output, understanding what happens at each stage and why each component is necessary.

## 6.3. Lesson 3: Hands-On Projects and Real-World Deployment

The third lesson moves from understanding to building. Students assemble a complete invoice extraction pipeline using the concepts from the first two lessons. They handle real challenges such as multi-page invoices, scanned images that require text recognition, and invoices where some fields are absent.

Students also build an intelligent chatbot over a collection of past invoices. An employee can ask questions such as "How much did we pay TechParts Ltd. in the first quarter?" or "Which invoices from Elektro GmbH are unpaid?" The chatbot uses RAG to retrieve the relevant invoice records and the language model to formulate a clear answer.

The lesson ends with a discussion of deployment: how to make the system available to the accounts-payable team, how to monitor its accuracy, and how to improve it over time.

# Chapter 7. Practical Outcomes

## 7.1. Automating Structured Data Extraction

Students leave this course able to design and build systems that extract structured data from unstructured documents. This skill applies far beyond invoices. The same approach works for purchase orders, contracts, medical records, insurance claims, and any other document where key information must be identified and organized.

The business value is direct and measurable: hours of manual effort are replaced by seconds of automated processing, and the error rate drops to near zero for well-formed documents.

## 7.2. Building Intelligent Knowledge Retrieval Chatbots

The RAG architecture enables a new category of application: a chatbot that genuinely knows the contents of a large document collection and can answer questions about it in natural language. Unlike a keyword search, this chatbot understands the question and finds the answer even when the words in the question do not appear verbatim in the documents.

For an accounts-payable team, this means instant answers to questions that previously required opening and reading multiple files. For a legal team, it means instant answers about contract terms. For a customer support team, it means instant answers drawn from a product knowledge base.

## 7.3. Deploying Real-World Business Process Automation

The deepest outcome is the ability to deploy working solutions that remove manual effort from business processes. Fewer errors mean fewer corrections. Faster processing means faster payments and better supplier relationships. Consistent output means better data quality in downstream systems.

These are not theoretical improvements. They are measurable outcomes that businesses value and that demonstrate the practical impact of the technologies covered in this course.

# Chapter 8. Summary

Manual invoice processing is a clear and relatable example of a problem that affects nearly every organisation. It is time-consuming, error-prone, and does not scale. RAG solves this problem by combining the semantic understanding of a Large Language Model with the targeted retrieval power of a vector database.

The key insight is that the system does not need to know in advance how each supplier formats their invoice. It needs to understand what an invoice is and what information it contains. The language model provides that understanding. The vector database provides the ability to find the right section of the right document quickly. Together they turn an unstructured PDF into clean, validated, structured data in seconds.

The technologies—Large Language Models, vector databases, and integration frameworks like LangChain—are the building blocks of a wide class of intelligent document processing applications. Mastering them through the concrete example of invoice processing gives students a foundation they can apply immediately in industry.

# Chapter 9. Quick Self-Check Questions

Use these questions to test your understanding before moving to the next lesson. There are no trick questions. Each one maps directly to a concept covered in this document.

## 9.1. Foundational Level

These questions check whether you have understood the core ideas. You should be able to answer each one in two or three sentences without referring back to the text.

- **What is the main reason manual invoice processing fails when business volume grows?** - Think about what happens when the number of invoices doubles. Does the solution scale? What does that mean for cost and accuracy?
- **A company receives invoices from 200 different suppliers. Each supplier uses a different layout and field naming convention. Why would a rule-based script that looks for specific keywords fail in this situation?** - Consider what happens when one supplier writes "Total Due" and another writes "Montant Total". What does a keyword rule do when it does not find the word it is looking for?
- **What does RAG stand for, and what do the two parts—retrieval and generation—each contribute to the system?** - Try to explain each part separately before describing how they work together.
- **In the context of RAG, what is a chunk, and why is the invoice split into chunks rather than processed as a single whole document?** - Think about the context window limit of a language model and the precision of search when dealing with small focused sections versus an entire document.
- **What is the role of the vector database in the RAG architecture?** - Describe what it stores, what kind of question it can answer quickly, and at which point in the pipeline it is involved.

## 9.2. Intermediate Level

These questions require you to connect concepts, reason about design choices, and apply your understanding to slightly different scenarios. A good answer goes beyond restating definitions.

- **The same embedding model must be used in both the ingestion pipeline and the query pipeline. What would go wrong if a different embedding model were used at query time, even if the second model were of higher quality?** - Think about what a vector represents and what it means for two vectors to be "close" to each other. Would vectors from two different models occupy the same numerical space?
- **During the query pipeline, the system retrieves the top-3 most relevant chunks from the vector database and passes them to the language model. What is the risk of setting this number too low (for example, top-1)? What is the risk of setting it too high (for example, top-20)?** - Consider a multi-page invoice where the vendor name appears on page 1 and the payment terms appear on page 3. What happens if only one chunk is retrieved? What happens if too many irrelevant chunks are included in the prompt?
- **A student argues: "We do not need RAG for invoice processing. We can just paste the entire**

invoice text directly into the prompt and ask the LLM to extract the fields." Under what conditions is this student correct, and under what conditions does this approach break down? - Think about invoice length, cost per token, and what happens when a company needs to process thousands of invoices simultaneously.

- **The ingestion pipeline and the query pipeline are described as separate. What practical advantage does this separation provide for a business that receives invoices continuously throughout the day?** - Consider whether a new invoice needs to be fully ingested before a query about a previously ingested invoice can be answered. What does this mean for system availability?
- **RAG is described as applicable beyond invoice processing—to contracts, resumes, medical records, and customer support knowledge bases. What property do all of these use cases share that makes RAG a suitable solution for each of them?** - Think about what these document types have in common in terms of volume, format variability, and the nature of the information that needs to be extracted or retrieved.